

# Personal Portfolio

In this tutorial, we'll use HTML and CSS to create a basic website about yourself. By completing this tutorial, you will learn how to use basic HTML tags, semantic tags, how to apply CSS from an external file, and how to use online fonts.

Here is a partially completed version of the project: [Personal Portfolio](#)

By developing this project, you will be able to:

1. Describe the importance of creating a sketch before developing a website
2. Use basic HTML tags to:
  - a. Add paragraphs `<p>`
  - b. Add line breaks `<br>`
  - c. Add images ``
  - d. Add Links `<a`
  - e. `href="example.com">`
  - f. Bold text using `<strong>`
  - g. Add comments `<!-- -->`
  - h. Create an ordered and unordered list
  - i. Create a basic table with data `<table>`
  - j. Create `<div>` containers
3. Use Semantic tags to:
  - a. Create header, footer, and side sections.
4. Apply inline, internal, and external CSS:
  - a. Use classes and id to apply style
  - b. Change background and text color
  - c. Create a navigation menu
  - d. Center text and content
  - e. Use online fonts
5. Embed a Google Form within an iframe



## Planning!

Take a moment to think about our goal for this project and how we would achieve it.



- a) When designing a website or web page, it is helpful to create a mockup or wireframe. This is especially helpful when working as a part of a team since all team members can have the same visual reference.
- b) Identify the sections and elements that you will need (images, navigation, links, footer, etc.)
- c) Create the folder structure that you'll use for the website.

## Lesson 1: Planning and Website Mockup

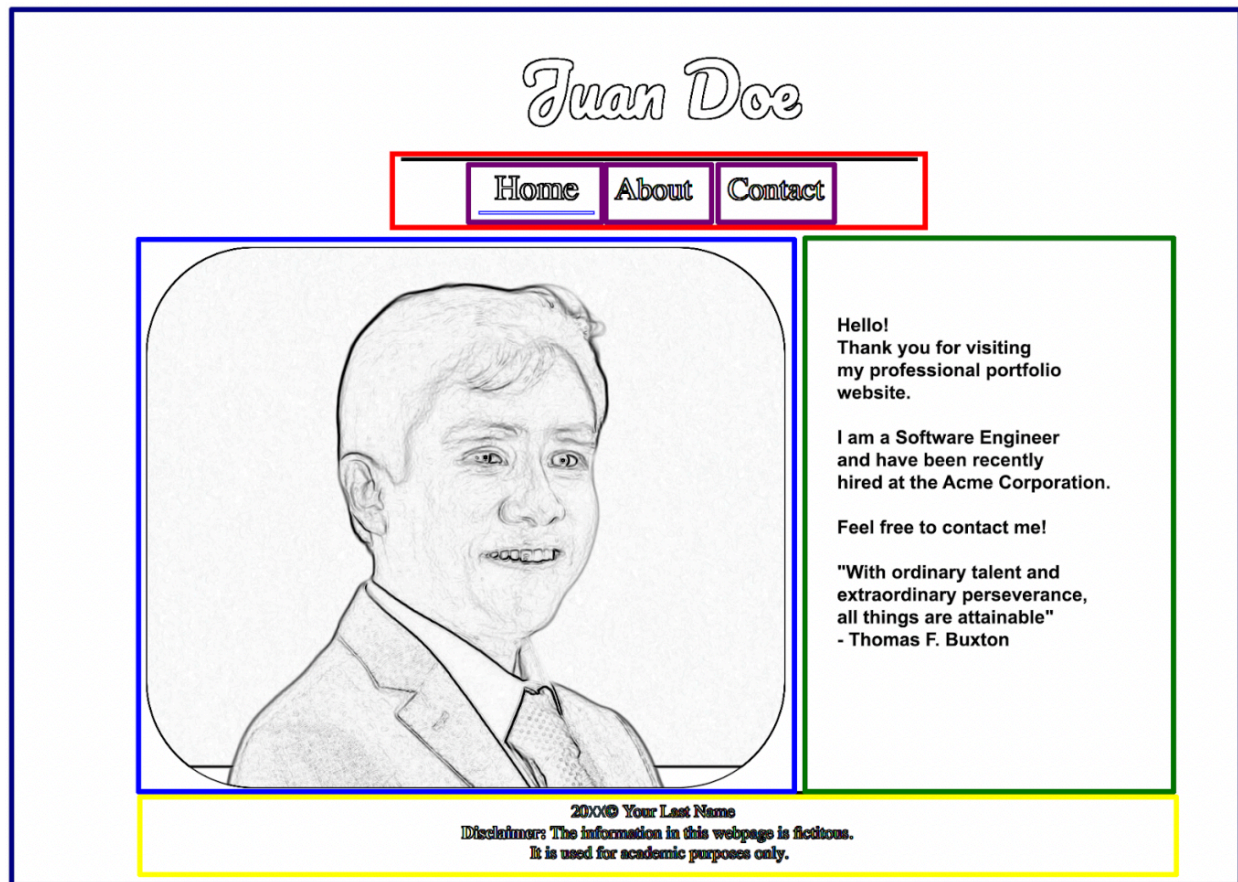
Before we start writing any code for our website, it's best to take some time to decide what we want it to contain and how to structure it.

For our portfolio home page, we want to have:

- Website title
- Navigation
- A professional portrait
- A little blurb welcoming the user
- A footer with copyright info

Next, we need to think about how to lay out the web page. We can create a basic mockup of the website's structure on paper or something more sophisticated, like Photoshop.

Here is the basic mockup of the Home Page we will be creating:



It's handy to imagine web pages as a **set of rectangular containers** (or boxes) that can be nested inside of each other. In this example, we see the **body** (the **outermost rectangle in dark blue**) wrap the entire website. Inside the body, we have the **navigation** (in red), the **portrait** (in light blue), **blurb** (in green), and the **footer in yellow**. Notice the navigation is further broken down into **individual links** (purple).

By placing all of these elements into their own containers, or **semantic** and **<div>** tags, we can position and style them individually. Also, by separating them out, we can easily make changes to one element without disturbing the rest of the web page.

## Lesson 2: Preparing the Folder Structure

2.1 Create a **lab1** folder on your local computer, inside your “**labs**” folder

2.2 [Download and uncompress the zip file](#) containing the starting point for our website

2.3 Move the uncompressed content into the **lab1** folder

## Lesson 3: Scripting your First Page

3.1: Open your file called “**index.html**” :

Let’s analyze its current content:

- **<!DOCTYPE html>** (line 1) is required and tells the browser to treat the file as a standard HTML 5 file,
- **<head>** (line 4) Defines information about the document, such as title, author, etc.
- **<!-- -->** (lines 5 to 9) Defines a comment
- **<body>** (line 12) Defines the document's body
- **<footer>** (line 16) Defines the **footer** Semantic element

```
1 <!DOCTYPE html>
2 <html>
3
4 <head>
5 <!--
6     This is the head section.
7     It includes optional informac
8     It must include the charater
9 -->
10 </head>
11
12 <body>
13
14
15
16 <footer>
17     <!-- Your last name and y
18 </footer>
19
20 </body>
21
22 </html>
```

3.2: Update the **<head>** section

Two HTML tags should always be included as part of the Head section:

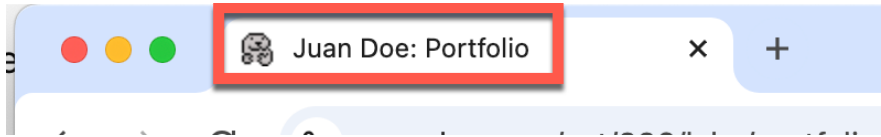
**<meta charset="UTF-8">** and **<title></title>**.

Include these lines within the Head section (use your name instead of “Juan Doe”)

```
<head>
  <meta charset="UTF-8">
  <title> Juan Doe: Portfolio </title>
</head>
```

The “charset” line specifies the character encoding to be used by the browser. UTF-8 is Unicode, which is the most comprehensive character set currently available.

The content of the <title> tag will NOT be displayed within the web page itself. If you save the document and open it in a browser, you’ll realize that the title is shown as part of the tab of the web page. But the content of the page is blank.



The <head> section will also include the CSS and JavaScript files. We’ll add CSS later on.

**Note:**



HTML tags are NOT case sensitive. However, **it’s best practice** to stick with either upper or lower case throughout the website to keep consistency.

### 3.3: Add the header and navigation within the **body** element.

All the content that will be displayed within a web page must be included within the <body> tags. Add the following code to the web page, right below the <body> tag:

```
<body>
  <header>
    <h1> Juan Doe </h1>
  </header>
  <hr>
  <nav>
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
    <a href="contact.html">Contact</a>
  </nav>
```

- **<header>** defines a semantic section as a header

- **<h1> - <h6>** defines HTML headings; gets smaller as the number gets higher
- **<hr>** defines a thematic change in content, visually represented by a line or “horizontal rule”. Notice that the **<hr>** tag is an **empty** HTML element (it has no content; therefore, there is no closing **</hr>**).
- **<nav>**: defines a semantic section as a navigation menu.
- **<a>** creates a hyperlink (when clicking on “Home” it will open the file “index.html”, when clicking on “About” it will open “about.html”, and so on).

### 3.4: Add an image and content

Add the following code right below the closing **</nav>** tag added in the previous step.

```
18     </nav>
19
20     <br><br>
21
22     <main>
23
24         <figure>
25             
26         </figure>
27
28         <div>
29             Hello! <br />
30             <p>Thank you for visiting my professional portfolio website.</p>
31
32             <p>I am a Software Engineer and have been recently hired at
33             the Acme Corporation.</p>
34
35             <p>Feel free to contact me!</p>
36
37             <br><br>
38
39             <em>"With ordinary talent and extraordinary perseverance,
40             all things are attainable"</em>
41             <br>
42             - Thomas F. Buxton
43         </div>
44
45     </main>
```

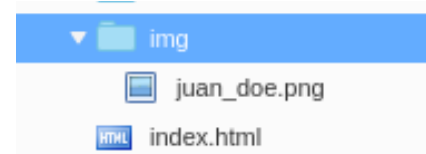
(The line numbers might not match yours; they’re just for reference only)

- **<br>** (line 20) creates a line break (carriage-return). It doesn’t use a closing tag.

- **<main>** (line 22) defines a semantic section in a document that is expected to include the main content
- **<figure>** (line 24) defines a semantic section that is expected to include an image
- **<img>** (line 25) displays an image. It doesn't need a closing tag. Uses two attributes: "**src**", which is the image source, and "**alt**", which is alternative text for visually impaired users.
- **<div>** (line 28) defines a container within the document
- **<p>** (multiple lines) defines a paragraph
- **<em>** (line 39) defines emphasized text, displayed in *italics*.

**Note:** The 'img' folder contains a default image of "juan\_doe", but you should upload your own image.

**File names are case sensitive.**



## Bad Practice!

The `<img>` tag provides "width" and "height" attributes that allow you to modify the display size of the image; however, these attributes do NOT change the actual image size.



So, if you upload a big image that is 2,000 x 2,000 pixels, it will still take a lot of bandwidth and time to be downloaded, even if the values of the "width" and "height" attributes are 20.

The best practice is to change the actual image size using image editing software.

### 3.5: Create the Footer

The footer will allow us to display information about our website. And a disclaimer telling the user that this information is for academic purposes only. Please, include a footer on each assignment.

```
47     <footer>
48         <hr>
49         CST336 Internet Programming. 20XX&copy; Your Last Name <br>
50         <strong>Disclaimer:</strong> The information in this webpage is fictitious. <br>
51     </footer>
52
```

- **&copy;** (line 49) creates a copyright symbol.  
(Update the year and your last name)

## Let's see our page!

Since we have added just HTML code, if you're working on your local computer, you can just double-click on the file name to open it in any browser of your preference.

### Note:



“**index.html**” is a special file name in web servers, when a file is called “index.html”, you don't need to type it as part of the URL.

When opening the page in a web browser, you should be able to see something like the screenshot to the right.

So far, we have added the content and also the web page structure (the semantic tags).

Feel free to keep customizing the content of this page. In addition to replacing the image and name, you should also change the quote and the text.

## Juan Doe

[Home](#) [About](#) [Contact](#)



Hello!

Thank you for visiting my professional portfolio website.

I am a Software Engineer and have been recently hired at the Acme Corporation.

Feel free to contact me!

*"With ordinary talent and extraordinary perseverance, all things are attainable"*  
- Thomas F. Buxton

---

CST336 Internet Programming. 20XX© Your Last Name  
**Disclaimer:** The information in this webpage is fictitious.



We can proceed now to style the page by adding CSS rules.

## Lesson 4: Adding CSS Styles to Home Page

CSS (Cascading Style Sheets) is the code you use to style your webpage (i.e., change color, fonts, borders, spacing, etc.) There are three ways to apply styles:

- **inline,**
- **internal, and**
- **external.**

### 4.1 Applying Inline Styles.

Inline styles are included within the HTML elements themselves.

Let's change the background color of the entire web page by using an inline style.

Modify the **<body>** tag in this way:

```
22 <body style="background-color:black; color:white">
```

Save and open the file.

You should see that it has a black background and white text now.

While you might need to use inline styles on rare occasions, its use is highly discouraged because it makes the code difficult to maintain.

Don't remove the inline style just yet, we'll remove it later on.

### Bad Practice!

It is bad practice to use inline styling as it clutters code. Also, you would need to make many changes in case you decide to update some styles. A better practice is to use external styles.



## 4.2 Applying Internal Styles.

Internal styles are included within the **<head>** section. They are especially convenient when the styles are intended for just one web page. Add the following code within the **<head>** section, right below the closing **</title>** tag:

```
17     <style>
18         body {
19             background-color: #000066;
20         }
21     </style>
22 </head>
```

Be sure to type the **<style></style>** tags correctly.

We have just added a CSS rule. The **selector** in this rule is the “body”, that is, the whole web page. The **property** is the background-color, and the value is a dark blue color represented by its hexadecimal value #000066, For more hexadecimal values, you can use [colorpicker.com](http://colorpicker.com)

However, if you save and open the document in a browser, you’ll notice that the background is still black! The reason is that the inline style takes priority.

A way to change the priority of inline style is by using the keyword: **!important**

Change the new style rule to read:

```
17     <style>
18         body {
19             background-color: #000066 !important;
20         }
21     </style>
22 </head>
```

If you save and open the page, you’ll see the new background color now.

As you may see, internal styles are an improvement over inline styles. However, when you have multiple pages using the same styles, internal styles are not the best option because you’d need to replicate the same CSS rules as many times as there are web pages. In those situations, it is more convenient to use an external style file.

### 4.3 Applying External Styles.

Let's start by removing the inline style created in step 4.1 for the `<body>` tag. Also, remove or comment out the CSS rule created in step 4.2:

```
<style>
  /* body {
    background-color= #000006 !important;
  } */
</style>
```

Notice that the syntax to comment out CSS code is different than for HTML code!

#### Tip



In most IDEs, you can press **Command + /** (or **Control + /**) to comment out lines of code.

Add the following line right below the `<title>` element.

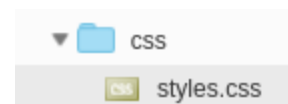
```
<title> Juan Doe: Portfolio </title>
<link href="css/styles.css" rel="stylesheet">
```

The `<link>` tag links this web page with an external CSS file (which we haven't created yet). The `href` attribute specifies the path to the CSS file. It can have any name but it must have `.css` extension. **Be aware that filenames ARE case sensitive in web servers.**

The `rel` attribute has the value of "stylesheet", which specifies that this external file is indeed a stylesheet. This attribute is required.

#### 4.3.1 Add a new file within the **css** folder and call it "**styles.css**".

You can use any other file name, such as "portfolio.css", however, the name must match the name used in the `<link>` tag.



#### 4.3.2 Style the body of the page. Add the following CSS rule to the **styles.css** file:

```
1 body {
2   background-color:black;
3   color:gray;
4 }
5
```

Save the CSS file and refresh the “Home” page in your browser. You should see that the page has a black background and the text is gray, not white.

## Troubleshooting:

If you don't see the styles applied to your web page, take the time to debug your code. Some potential reasons your CSS code might not work:



- 1) The code might have typos. 99% of the CSS problems are caused by **typos**. Make sure you typed everything correctly, double-check the CSS rules syntax, each line must end with a semicolon.
- 2) You might not be linking the external CSS file at all. Recall that file names are case sensitive. To ensure you are linking it properly, view your web page source in your browser and click on the link to your CSS file (all browsers provide a feature to view the source code).
- 3) You might have non-accepted characters in the CSS file. Your CSS file must not include any HTML tag, not even `<style>`. Must not include HTML comments either. Any odd character will break your CSS code.
- 4) In odd situations, your browser might cache the CSS style and ignore new style rules. To force the browser to take the new rules, just open the CSS file in your browser directly. Optionally, you can use these key commands to override the cache:  
Windows: CTRL + SHIFT + r  
Mac OS X: Command + Shift + r

### 4.3.3. Centering the Content.

Add the following CSS rule to center the content in all the semantic containers. Notice that you can apply the same rule to multiple HTML elements if you separate them with a comma.

```
6 | header, main, nav, footer {  
7 |   text-align: center;  
8 | }
```

Save the CSS file and refresh the web page. Now all content should be centered. Try removing any of the four semantic containers listed in the CSS rule and you should see that the content for that container is aligned to the left, which is the default alignment.

#### 4.4.4 Styling Navigation.

Type the code below to style the navigation menu. Pay attention to the syntax to add comments within CSS.

```
10  nav a { /* links within "nav" container */
11      padding:15px;
12      padding-bottom:4px;
13      font-size: 1.8em;
14      text-decoration: none; /* removes underline */
15      color: white;          /* by default, links are blue */
16  }
```

Save the file and see the changes. Take a minute to explore different values on each of the above properties.

Why do you think the selector of the menu links was “**nav a**” instead of just “**a**” ?



When you use an HTML tag as the selector of a CSS rule, the styles are applied to ALL elements with the same tag. So, if there was any other link within the page, it would have the same style as the navigation menu, which would look odd.

By using “**nav a**” as the selector, we are restricting the styles to just the links that are included within the “nav” container. Notice that there must NOT be a comma or any other character between “nav” and “a”, only a blank space.

Now, let’s add something fancy! The following CSS rule is using the “**:hover**” pseudo class, which means that the styles are going to apply only when hovering over that specific element. You can add the :hover pseudo class to any HTML element.

```
18  nav a:hover { /*pseudo class*/
19      border-bottom:2px orange solid;
20      font-size: 2em;
21      transition: .9s; /* transition effect */
22  }
```

#### 4.4.5 Positioning Content Side by Side.

Currently, the image is showing above the welcoming blurb but we really want them to be side by side. CSS provides several ways to create a multicolumn layout. A very popular way is using **Flexbox** (flexible box).

Follow these 3 steps to apply the Flexbox method:

- 1) Identify the elements that need to be next to each other. If you take a look at your HTML code, the image is included within the **<figure>** container and the welcoming blurb is included within a **<div>** container. So, the elements that need to be within each other are **<figure>** and **<div>**
- 2) Surround all elements that need to be next to each other within another tag... which we're already doing! As you can observe, the **<main>** tag surrounds these two elements. It didn't have to be "main", it could have been any other HTML element, such as a **<div>** tag.
- 3) Add the "display: flex" CSS rule to the surrounding tag

```
24     main {  
25         display: flex;  
26     }
```

Save the file and open it in a browser. The image and the welcome text should be displayed next to each other now, however, they're not centered. To center it content within a flex container, use the **"justify-content"** property, its value can be "center" or "space-evenly", try both!

```
24     main {  
25         display: flex;  
26         justify-content: center;  
27     }
```

#### Note:



Any element that you put inside a "flex" container will be displayed in a row. If there were more **<div>** containers within the **<main>** tag, all of them would be displayed next to each other.

Refer to [w3schools](https://www.w3schools.com) for more styling options

## It's Your Turn!



Before moving on, add more content and styles to accomplish the following:

- 1) Add the CSUMB logo at the bottom of your web page, within the footer.
- 2) Make the footer text smaller and with a different text color (your choice!)
- 3) In the navigation menu, the item corresponding to the current page **should stand out**. In other words, "**Home**" should stand out in the "index.html" page.

## Lesson 5: About Me Page

Let's expand our website beyond a single page!

**5.1:** Now that we have the content of our main page finished, let's create the about page. Create a new file and name it: **about.html**

**5.2:** Open the **about.html** file. Instead of typing the navigation and footer again, let's copy everything from **index.html**.

Delete everything within the `<main>` element, including the `<main></main>` tags. For this webpage will be using a "**<div>**" container just to demonstrate that you can use either a semantic tag (like `<main>`) or a `<div>` container with an id.

```
<div id="content">|  
  
</div>  
  
<!-- This is the footer -->
```

Save the file and now try the 'About' link in the menu. You will see we have the same menu and footer, but the content of the page is blank.

### Note:



The purpose of an “**id**” as the attribute of an HTML element is to identify that element to access it via CSS or JavaScript.

The id **must be unique** in the entire web page.

Think of it as a variable name: they are case sensitive and must not have any spaces.

**5.3:** On the **about.html** page we’ll display our programming experience with different languages. A good way to represent this kind of data in html is with a **<table>**.

Inside each **<table>** we need to use:

- **<tr>** to create rows
- **<td>** for individual pieces of table data (cells)

```
<div id="content">
  <!-- Create the table to hold experience data -->
  <table>
    <tr id="table-header">
      <td><strong>Programming Language</strong></td>
      <td><strong>Years Experience</strong></td>
    </tr>
    <tr class="table-row">
      <td>Java</td>
      <td>3</td>
    </tr>
    <tr class="table-row">
      <td>C++</td>
      <td>2</td>
    </tr>
    <tr class="table-row">
      <td>PHP</td>
      <td>1</td>
    </tr>
  </table>
</div>
```

You will notice we added the id “**table-header**” to the header **<tr>** tag and “**table-row**” to the other **<tr>** tags. This will allow us to style them independently of each other in the stylesheet.

**5.4:** In addition to the programming experience table, let’s include a little about our interests.

We can use a list for this purpose. Start by creating a new unordered list using the **<ul>**.



Inside the list tags, we create new list items with `<li>`. Customize this list with your own interests!

```
<ul>
  <li>Video games: I own four consoles and like all kind of games but my favorite is The Legend of Zelda</li>
  <li>Soccer: I am part of a team and we play every Saturday. I'm the goalie</li>
  <li>Music: I love all kind of music but my favorite is R&B and Pop.</li>
  <li>Programming: I spend at least two hours every day using and learning programming languages </li>
</ul>
```

Save the file and open it in a browser window. You'll notice that the table and the unordered list are NOT centered. You might be thinking that they could be centered by applying the "text-align:center" CSS rule to #content, but they won't.

#### Note:



To center a block element, such as a **HTML table** or a **div** container, you must assign the value of "auto" to the properties: margin-left and margin-right:

```
margin-left: auto;
margin-right: auto;
```

Optionally, you could use the shortcut version:

```
margin:0 auto;
```

When you provide only two values to margin, the first is applied to the top and bottom, and the second is applied to the left and the right

**5.5:** Add the following CSS rules to the **styles.css** to center the table:.

```
table {
  margin: 0px auto;
}
```

**5.6:** We can make it a touch more legible by adding **10px of padding** and more consistent with the homepage by increasing the **font-size to 1.3em**.

In order to make the header of the table stand out, we can specifically target `<strong>` tags inside of `<td>` tags and provide additional styling to them. This is done by putting the parent tag followed by the child tag. In our case: `'td strong'`.

```
td {  
    padding-right: 10px;  
    font-size: 1.3em;  
}  
  
td strong {  
    color: orange;  
}
```

**5.7:** Using the **class** and **id** attributes we put on the rows of our table, we can change the style of them independently. The hash “#” character means id, the period “.” means class. Feel free to use your own colors!

```
#table-header {  
    background-color: #351a00;  
}  
  
.table-row {  
    background-color: #753900;  
}
```

#### Note:



To style our table, we used an **id** for the header and a **class** for the row. This is because we have a single header, whereas there are multiple rows of the table.

As a rule:

- Use **id** to identify a single element to style.
- Use **class** to style a group of elements (at least two)

**5.8:** Our list of interests needs a touch of styling as well. Use `margin: 0 auto;` to center the whole block. You must also specify a width.

```
ul {  
    width: 500px;  
    margin: 0 auto;  
    text-align: left;  
}
```

## How would you do it?

Let's say you wanted to add color to each of the hobby nouns (the part before the colon), how would you do this? Given what you know about CSS and HTML, which tags would you style?



**5.9:** In order to style some element on a page, it needs to be contained within HTML tags. If you want to style just the hobby nouns on the list (video games, soccer, etc), we need to put tags around them. For this, we will use the `<span>` tag with a class we can reference in the stylesheet.

### about.html

```
<ul>
  <li><span class="hobby">Video games</span>:
  <li><span class="hobby">Soccer</span>: I am
  <li><span class="hobby">Music</span>: I love
  <li><span class="hobby">Programming</span>:
</ul>
```

### styles.css

```
.hobby {
  color: yellow;
}
```

### Note:



So if we need to section off a part of the page content to style, like the hobbies above, why not use a `<div>` like we did before? This is because `<div>` is a **block** element and `<span>` is an **inline** element.

- Block elements are not friendly and do not allow other elements to sit next to them. In other words, other elements will be displayed in a different line.
- Inline elements are very friendly and will gladly sit next to other elements.

You can change any element to be inline or block by using the **'display'** property in your stylesheet.

### The result:

# Juan Doe

Home About Contact

| Programming Language | Years Experience |
|----------------------|------------------|
| Java                 | 3                |
| C++                  | 2                |
| PHP                  | 1                |

- **Video games:** I own four consoles and like all kind of games but my favorite is The Legend of Zelda
- **Soccer:** I am part of a team and we play every Saturday. I'm the goalie.
- **Music:** I love all kind of music but my favorite is R&B and Pop.
- **Programming:** I spend at least two hours every day using and learning programming languages

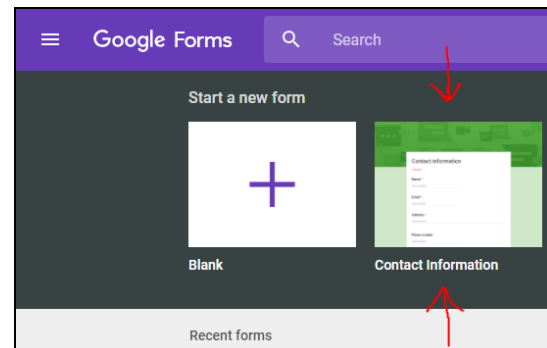
CST336 Internet Programming. 20XX© Your Last Name  
**Disclaimer:** The information in this webpage is fictitious.

## Lesson 6: Contact Page

**6.1:** Next, we can move on to the Contact page. Like the **About** page, we will copy over our **index.html** and remove everything inside **<main>** element.

**6.2:** For this page, we're going to use a [Google Form](https://docs.google.com/forms) to handle the contact feature. Later in the semester we will create our own forms, but that requires server programming. Besides, Google Forms are super slick and work great!

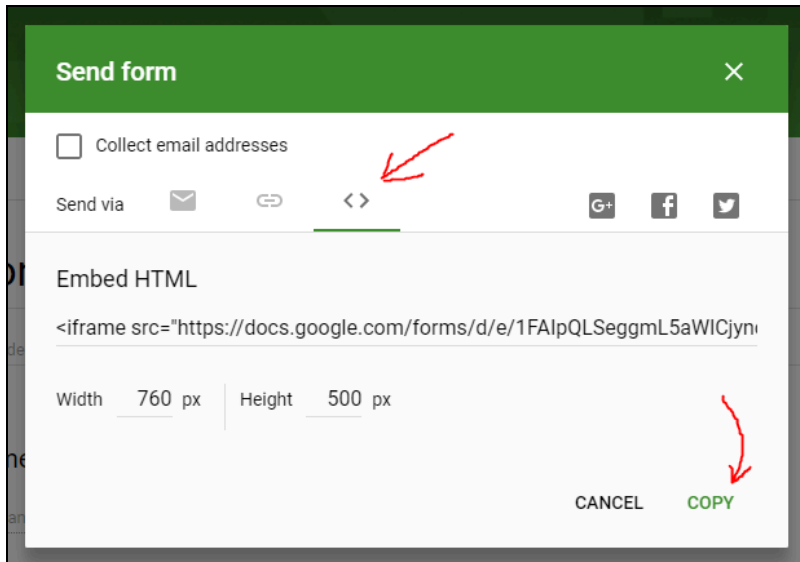
Go to <https://docs.google.com/forms> while logged into your Google account and select the **Contact Information** template from the gallery at the top.



**6.3:** On the form edit page, make any changes you would like and when you're done, click the **Send** button in the upper right-hand corner.



**6.4:** When the Send Form dialog pops up, click the menu option that looks like HTML tag brackets (< >) and click the copy button to add the **<iframe>** html text to the clipboard.

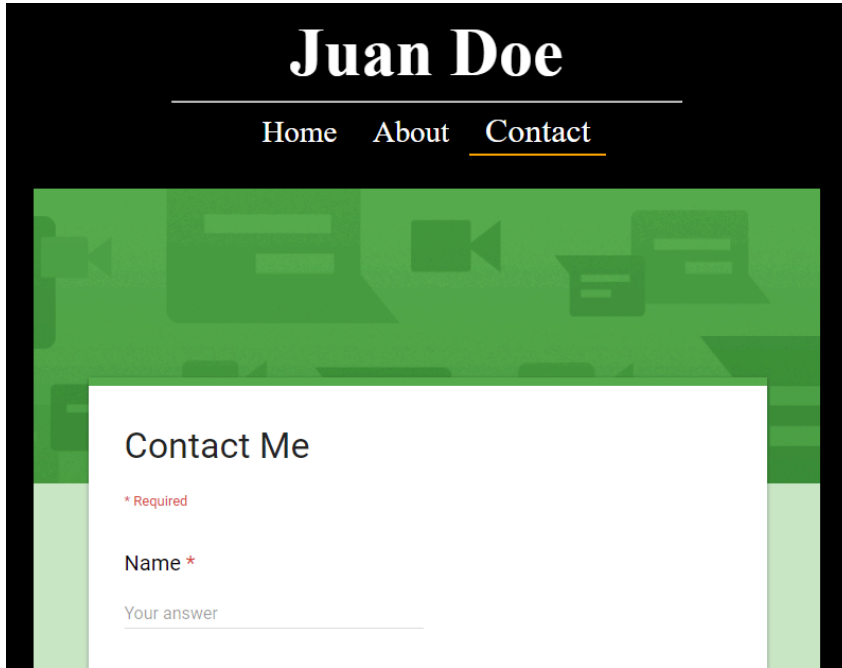


The HTML tag **<iframe>** is used to embed a “frame” or window of one website into another. The Width and Height properties allow you to select how big the window frame is.

**6.5:** Now that we have the text for our iframe form copied, let's paste it into our contact page.

```
<!-- div with an id called content which we will re
<div id="content">
  <iframe src="https://docs.google.com/forms/d/e/
</div>
```

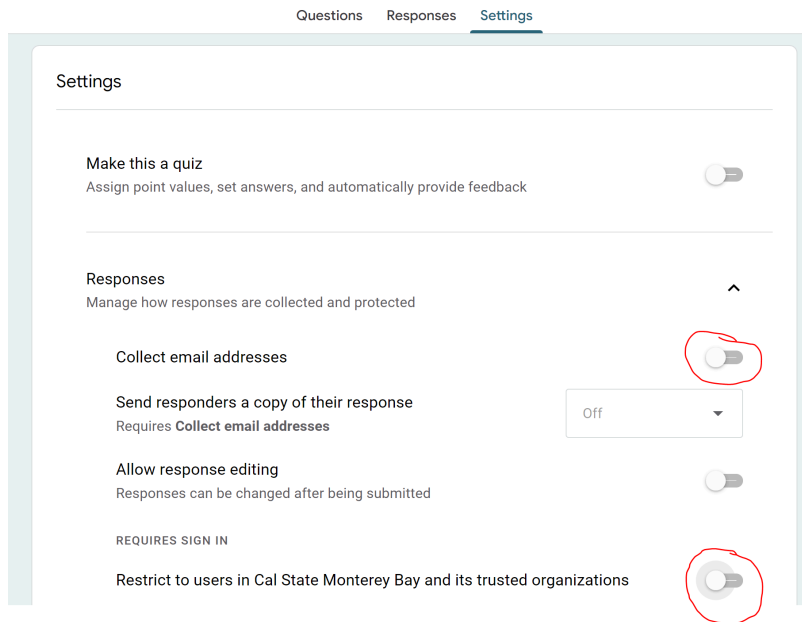
The result:



The screenshot shows a web page with a black header. At the top, the name 'Juan Doe' is displayed in a large, white, serif font. Below the name is a horizontal line with three navigation links: 'Home', 'About', and 'Contact'. The 'Contact' link is underlined in yellow. The main content area has a green background with a faint, abstract pattern. In the center, there is a white rectangular form titled 'Contact Me'. Inside the form, there is a red asterisk followed by the word 'Required'. Below this, the label 'Name' is followed by a red asterisk. Underneath the label is a text input field with the placeholder text 'Your answer'.

**Note:** If you create the Form while signed into your CSUMB account, your Form will only be available to users with a CSUMB account. Users who don't have a CSUMB account or are not logged in will not be able to see your Form. To change the settings, go to your Google Form and make the following two changes within the **Settings** tab (refer to screenshot below).

- 1) Disable "Collect email addresses."
- 2) Disable "Restrict to users in CSUMB."



## Lesson 7: Using Online Fonts

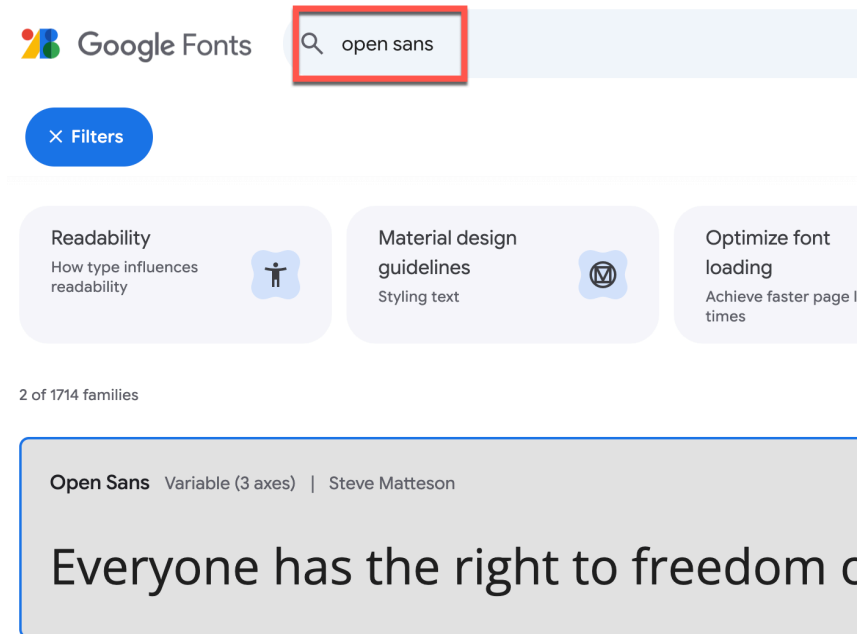
Let's get some stylish fonts!

Our website is mostly complete, but the header could use something to help it stand out. One option would be to use an image, but consumes more bandwidth and doesn't allow for scaling. A better way is to use a different and nicer font than the rest of the web page.

Not all of the fonts that you have on your local computer are available in all browsers, though. Only the most common fonts are available (e.g., Arial, Time News Roman, Verdana). If you want to use a fancy font that can be displayed in any web browser, a good alternative is using an external web font.

To improve readability, you should only use fancy font families in main headings. For the actual content, you should ideally use a sans-serif font. [This document](#) provides a list of the top ten Google Fonts, which includes "Open Sans", "Roboto" and "Montserrat", among others. Let's use one of them!

1. Open [Google Fonts](#) in a new browser window.
2. Type "Open Sans" in the "Search Fonts" text box.  
It should find and display the "Open Sans" sample box.

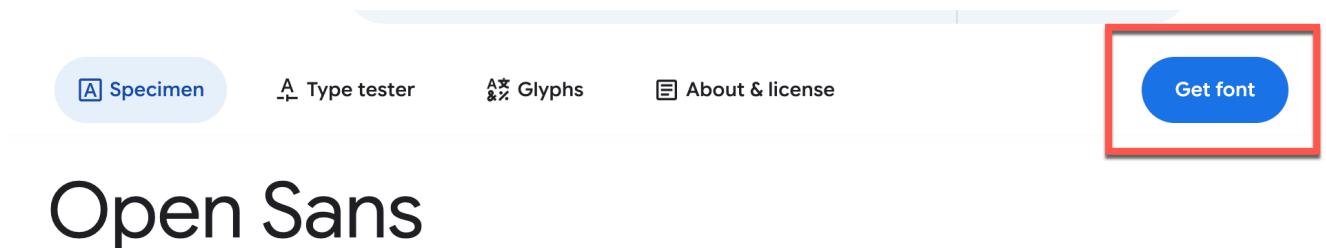


3. Click on the “Open Sans” sample box



It will open the “Open Sans” font web page with the list of all styles available.  
Most font families include a variety of thicknesses, from light to extra bold.

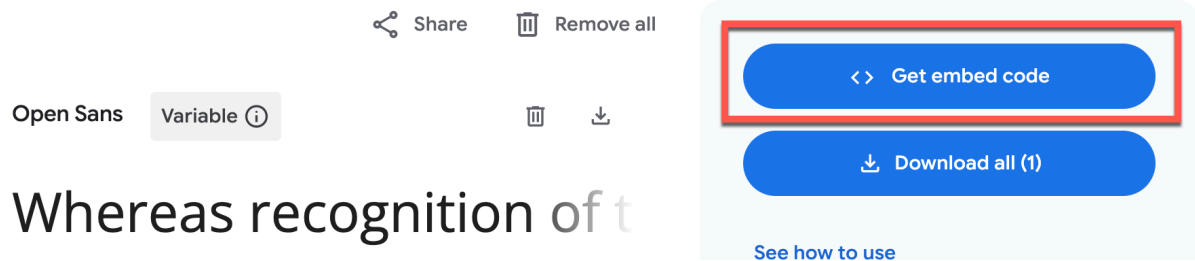
4. Click on the “Get font” button



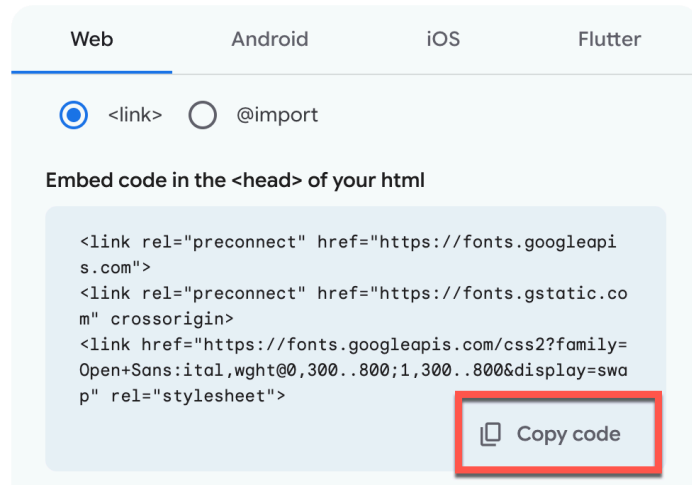


- It will inform you that you've selected one font family. Click on the **"Get embed code"** button

## 1 font family selected



- Then, we need to copy the HTML code generated to link the family types. Within the **"Web"** tab, click on the first **"Copy code"** icon located at the bottom right corner. This will store the HTML code in the clipboard and is ready to be pasted into the **index.html** file.



- Open the **"index.html"** file and paste the HTML code within the **<head>** section, right below the **<title></title>** tags. Three new **<link>** tags should be pasted (see screenshot below)

```
<!DOCTYPE HTML>
<head>
<meta charset="UTF-8" />
<title>How to Buy from a Vending Machine Job Aid</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Open+Sans:wght@400;800&display=swap" rel="stylesheet">
<link rel="stylesheet" type="text/css" href="css/jobaid.css" />
```

- Next, we need to specify the HTML element that will use the font family. First, we need

to go back to Google Fonts and copy the CSS rule located below the “Open Sans: CSS class for variable style” section. Click on the “Copy code” icon.

#### Open Sans: CSS class for a variable style

```
// <uniquifier>: Use a unique and descriptive class name  
// <weight>: Use a value from 300 to 800  
  
.open-sans-<uniquifier> {  
  font-family: "Open Sans", sans-serif;  
  font-optical-sizing: auto;  
  font-weight: <weight>;  
  font-style: normal;  
  font-variation-settings:  
    "wdth" 100;  
}
```

 Copy code

Since we want to use the same font family for the whole document, we need to apply it to the “**body**” element. Open the “**styles.css**” file and paste the CSS code at the bottom of the file.

a. Remove the first two lines with comments:

```
// <uniquifier>: Use a unique and descriptive class name  
// <weight>: Use a value from 300 to 800
```

b. Replace “.open-sans-<uniquifier>” with “**body**”

c. Replace **<weight>** with a number between 300 to 800, as indicated in the comment

```
body {  
  font-family: "Open Sans", sans-serif;  
  font-optical-sizing: auto;  
  font-weight: <weight>;  
  font-style: normal;  
  font-variation-settings:  
    "wdth" 100;  
}
```

9. Reload your web page, and you should now see the new font family being applied.

To use a fancy font family for the main heading, you need to follow the same steps: locating the font to use, adding it to the list of selected families, copying/pasting the <link> tags into the “**index.html**” file, and copying/pasting the “font-family” property in the **styles.css** file, within the “**h1**” element, since it should only affect the main heading.